

This book kept forgetting how to write itself. That sounds more mystical than it was. I would work on a chapter with an agent, reject a certain kind of edit, explain why I rejected it, and eventually get something better. A few days later we would start another chapter and the same failure would return. The prose became cleaner in exactly the wrong way. Wandering sentences disappeared. Strange jokes were replaced by respectable ones. Arguments broke into tiny paragraphs that looked dramatic from across the room and exhausted me when I actually read them.

So I would say things like:

Don't kill the wandering. Don't turn every idea into a slogan. Preserve the weird joke if it is carrying the argument. "More polished" is not automatically "more mine."

The agent would improve. Then the context would end.

We were reenacting, on a ridiculous scale, the problem Chapter 5 had just spent several thousand years describing. A society can know something none of its members knows alone. Fine. But if the society survives, another question appears:

How does what it learned yesterday change what it does tomorrow?

The obvious answer is memory. Save the conversation. Increase the context window. Keep a notebook. Put every decision into a database. That helps. It is not enough.

A transcript remembers what happened. An institution has to remember **what was worth learning from what happened.**

Suppose I save the instruction "use longer paragraphs." That is a memory of one correction. It is also a future disaster waiting politely in Markdown. The lesson was never that long paragraphs are good. The failure was a particular editing process compressing exploratory prose into a rhythm that felt machine-produced. Sometimes the cure was a longer paragraph. Sometimes a shorter sentence. Sometimes the correct edit was to stop editing.

FORGETTING-BOOK ·
MARGIN-VIGNETTE

Margin vignette: an open manuscript whose ink is lifting off the page like steam from the earlier chapters while a hand rewrites the same marginal correction again; a faint ghost of the same correction visible twice before. Melancholy, wry.

What the next agent needed was not merely the instruction. It needed enough of the **reason, evidence, boundary conditions and failure history** to know when the instruction deserved authority.

That is closer to culture than memory.

Chapter 5 ended with the claim that **System 3 is science**: not science as a pile of papers, but as a civilization-scale arrangement that lets fallible minds accumulate knowledge while preserving routes through which reality can object.

Chapter 5 named the solution. This chapter opens the machine.

Science did not arrive with one clean design. Its philosophy is largely the history of people discovering failure modes in knowledge itself: theories that could explain anything, experiments that implicated several assumptions at once, paradigms that made research possible and then made alternatives hard to see, communities that confused consensus with independence, methods that became rituals, and institutions whose allocation of attention helped determine what could become known.

If System 3 is going to borrow from science, those arguments are not decorative philosophy. They are design reviews written a few decades or centuries early.

So the problem here is not simply how agents remember. It is:

How should useful experience become reusable behavior without turning yesterday's success into scripture?

That question has become practical because knowledge itself is starting to become a software artifact.

Three Ways to Tell a Computer What You Know

For most of computing history, if you knew how a process should work, you translated that knowledge into code.

A refund under €50 can be approved automatically. A payment over some threshold needs another check. A production deployment requires a test. A user without permission cannot read this table. Human knowledge becomes **if, else, functions, schemas, state machines and permissions**.

Andrej Karpathy calls this familiar world **Software 1.0**: humans write the behavior directly.

Machine learning changed the contract. Suppose I cannot state the rules that distinguish a fraudulent transaction from an unusual but legitimate one. I can give you examples. We choose data, a model and an objective, and optimization pushes useful behavior into weights. That is **Software 2.0**.

It gave us capabilities that explicit rules could never have scaled to, but much of the learned behavior disappeared from inspectable code. The fraud model “knows” things no engineer wrote down. We can evaluate it, probe it and retrain it, but there is no `fraud_rules.py` containing the organization's accumulated understanding of fraud.

Large language models created a strange third possibility. In his 2025 talk *Software Is Changing (Again)*, Karpathy called it **Software 3.0**: programs written substantially in natural language and interpreted by a model rather than a conventional compiler.

A model can read something like:

Review this experiment. Check instrumentation changes before inventing a causal story. If click metrics rise while orders stay flat, inspect position and price shifts before celebrating.

There is no deterministic function there. There is operational knowledge. A competent human can interpret it. Now a sufficiently capable model can too.

The change goes beyond “prompts are code.” **Knowledge itself can become versionable, composable and executable.**

An organization can write down a procedure, examples, scripts, counterexamples, diagnostic questions, evidence, tool instructions and boundaries. The model supplies enough interpretation that every clause does not have to become brittle symbolic logic before it can affect behavior.

The model begins to look less like the knowledge base and more like an **interpreter for knowledge artifacts**.

Knowledge engineering is back. It is carrying Markdown.

MARKDOWN-BRIEFCASE · SPOT

Small spot: a 1980s expert-systems consultant's leather briefcase, reopened; instead of logic rules it spills plain paper notes with hash-mark headings (illegible squiggles, not readable text). One robot peers in, delighted.

Knowledge Engineering Comes Back Wearing Markdown

The old dream of knowledge engineering was reasonable. Find experts. Extract what they know. Put it in a knowledge base. Let software reason with it.

The problem was that expertise is offensively reluctant to become a clean rule set.

Ask an experienced engineer how to diagnose a production problem and the answer is rarely:

IF latency > 300ms THEN database

It sounds more like:

Start with the dependency graph, unless the spike began exactly at deployment. If only one market is affected, check the traffic split before touching the database. The cache metric lies under failover, so ignore it when this alarm is red. And if the problem started after the Tuesday migration, ask Sam because there is a thing with the old serializer that nobody wrote down properly.

Traditional expert systems struggled because converting that practice into formal logic was expensive and brittle. Machine learning offered an escape: stop asking experts to explain themselves and learn patterns from data.

LLMs change the trade again. They can interpret prose, examples, scripts, diagrams and partially structured instructions. Expertise still has to be captured, but it no longer has to become perfect logic before the computer can use it.

Now the hard questions are different. Whose practice gets written down?

Which version applies here? What happens when two experts disagree? How does a lesson lose standing? When does a local workaround become a global rule? Which knowledge should enter the working context now, and which should remain in the archive?

Those are no longer edge cases. They are the engineering surface.

Agent systems were already moving in this direction. Repository instructions, `AGENTS.md`, skills, tool descriptions and context engineering all treat useful procedural knowledge as something external to the model but available to it at runtime. A skill can survive the session that produced it. Increasingly the worker can change while the operating knowledge remains.

That matters because frontier models already know Python, statistics and enormous amounts of public technical culture. What they do not automatically know is why *your* company refuses to deploy on Friday, which metric has been misleading everyone since 2023, why an elegant architecture in the wiki was abandoned, or why Alberto should never again be asked to investigate penguins. Organizations run on this layer of weirdness.

Some belongs in code. Some belongs in data, tools and evaluators. A surprising amount is **situated procedural knowledge**: what to check first, which shortcut is dangerous, which source has standing, when the normal process does not apply, and what “good” means here rather than on a generic benchmark.

For most of history, people acquired this by hanging around people who had already been injured by the relevant mistakes.

Now more of it can become software. Which is exciting right up until we create prompt spaghetti at civilizational scale.

A saved instruction is too small a unit.

From Skill to Pattern

Christopher Alexander's *A Pattern Language* was about towns, buildings and recurring design problems. A pattern was not simply a commandment. It named a situation, the forces that made it difficult, a response that had repeatedly worked and the consequences of using that response.

That abstraction fits agent knowledge almost suspiciously well.

Consider:

Never use regex on nested syntax.

It has the reassuring shape of wisdom and the inconvenient property of being false.

Regex is perfectly good for many small extraction tasks. A parser may be absurd overhead for a five-line format. The useful lesson is that recursive structure creates characteristic failure modes for flat pattern matching; those failures become harder to see as syntax grows; and beyond some point a parser becomes cheaper than maintaining an increasingly heroic regular expression written at 2 a.m.

That is closer to a pattern.

A saved instruction remembers **what somebody said**. A pattern tries to remember **what kept happening**.

The book has already accumulated patterns whether we called them that or not.

Immutable Harness: when autonomy makes the solution fluid, keep the evaluation boundary harder to change than the thing being evaluated.

Independent Evaluators: when one judge can be gamed, create genuinely different sources of pressure rather than five copies of the same opinion.

Strategic Constraint: when an easy path keeps absorbing the search, remove it long enough to expose another part of the possibility space.

The useful part is not the slogan. Each pattern contains a recurring situation, a tension and a reason.

PATTERN-TOWN · CORNER-
BLEED

Corner scene: a small hillside town where each building visibly answers a recurring situation — a shaded corner where two paths meet, a window seat catching afternoon light, a market arcade — with faint pencil construction lines still showing, as if the town remembers why each form exists.

But Chapter 5 made us suspicious of inherited procedure. Civilizations do not merely accumulate good practices. They accumulate ritual, prestige, local workarounds and procedures that have outlived the world that justified them.

If Pattern Language is going to become cultural memory for agents, a pattern needs to know more about itself. This is where philosophy of science finally earns its API.

Popper: A Pattern Needs a Way to Lose

Suppose the editing agent concludes:

Use longer paragraphs in this book.

Karl Popper would immediately ask the rude question: what could happen that would make us stop believing this lesson?

A useful theory exposes itself to observations that could have gone differently. If every outcome can be narrated as success, the theory has arranged the game so it cannot lose.

Persistent agent knowledge needs the same property. A pattern should retain an **exposure path**: a test, observation, user reaction, proof obligation or downstream consequence capable of weakening it. “Use longer paragraphs” might lose standing if reader tests show comprehension falling, if another chapter becomes monotonous, or if the original failure disappears after the editing process changes.

The pattern should remember not only what worked, but **how the world could show that the lesson stopped working**.

Then Pierre Duhem and W. V. O. Quine ruin the simplicity.

Evidence rarely confronts one isolated belief. A failed experiment implicates a bundle: hypothesis, instrument, data, analysis, background assumptions. A failed pattern has the same problem.

Did the advice stop working? Was it retrieved in the wrong situation? Did the model change? Did the evaluator drift? Was the original success caused by something else? Did “longer paragraphs” merely correlate with the real change—more natural argumentative rhythm—without causing it?

Failure tells us that some part of the package deserves suspicion. It does

POPPER-CARD · MARGIN-
VIGNETTE

Margin vignette: a pattern written on a workshop card, hung on a nail — and beside it a small hammer on a cord, provided for breaking it. The card has a designated strike zone. No readable text; the affordance carries the idea.

not highlight the guilty line in yellow.

So reusable knowledge needs **archaeology**.

Where did this pattern come from? Which failures produced it? Which model and tools were involved? Which alternatives were tried? What evidence earned the lesson its standing? What assumptions were present?

A factual claim without provenance becomes rumor. A reusable practice without provenance becomes tradition.

Tradition is not automatically bad. It is simply difficult to debug.

Kuhn, Lakatos and Laudan: Defaults Need Rivals

Keeping every pattern permanently open for debate would be a beautiful philosophy and a terrible operating system.

Thomas Kuhn's most useful lesson here is not simply the phrase *paradigm shift*. It is the role of **normal science**. Productive communities need enough stability that they do not reopen their deepest assumptions every morning. A framework tells researchers what puzzles are worth solving, which instruments are legitimate and what kinds of answers count.

An agent culture needs the same economy. If a deployment procedure has survived hundreds of releases, the system should not rediscover it from first principles every Tuesday merely to prove that it remains intellectually alive.

Some decisions earn the right to become boring. The danger is that boring assumptions become invisible assumptions.

Anomalies need memory too. The release pattern works except in this market. The evaluator tracks human judgment except on this kind of creative task. The ranking heuristic works except every holiday season. The workflow works except that the critic now spends most of its time inventing reasons why the builder was right.

One anomaly is usually noise. Ten may still be noise. Eventually the exception list begins to look like the theory.

Imre Lakatos makes the problem harder in a useful way. We should often preserve **research programs**, not merely isolated ideas. A program has a history, a relatively stable core, auxiliary assumptions and a trajectory. One approach may currently be weaker but improving. Another may be winning mainly by adding patches around every failure.

Larry Laudan sharpens the practical consequence: **acceptance and pursuit are different decisions**.

I can believe method A is our best current default while still believing method B deserves another experiment.

Those are two questions:

- What should guide action now? - Where is another unit of investigation most valuable?

The distinction matters the moment culture becomes executable. The current winner gets retrieved more often. Because it gets used more often, it accumulates more successful cases. Those cases raise confidence. Higher confidence makes it even more likely to be retrieved.

The alternative receives less traffic and therefore less evidence. Eventually the system develops an impressive empirical record proving the thing it stopped comparing against.

No committee had to ban the alternative. The retrieval policy did it.

Hull and Kitcher: Who Gets the GPUs?

There is an overly clean way to draw a society of agents. Every agent is a box. Every box has an arrow. Everybody gets a turn to think.

Real institutions are not like that because **attention has a budget**.

Human science has telescope time, laboratory space, grants, journals, careers and prestige. AI research has datasets, deployment traffic, human reviewers, API quotas, clusters and GPUs. A theory's ability to generate evidence depends partly on whether the institution gives somebody the resources to investigate it.

This is not merely politics happening around epistemology. It changes the epistemic landscape itself.

Imagine research program A has ten thousand GPU-hours and program B has ten.

A can run ablations, train variants, investigate anomalies and produce beautiful graphs. B can produce a thoughtful paragraph about why it deserves more compute.

Six months later A has more evidence. Of course it does.

The evidence may be real. A may genuinely be better. But the institution has also helped create the asymmetry it later treats as evidence for further allocation.

David Hull and Philip Kitcher approached science partly through the division of cognitive labor, incentives, credit and the fact that investigators

do not all pursue the same thing for the same reasons. Researchers cooperate because they need one another's results and compete because priority, reputation, jobs and resources are scarce.

An agent society will have analogues of these structures whether or not we give them sociological names.

The scheduler is partly a funding agency. The memory system is partly an archive. The evaluator is partly a journal gate. The retrieval layer is partly a curriculum. The permission system decides who may touch which instrument. The compute allocator decides which hypotheses get enough opportunity to become well-tested hypotheses.

And the organization can remember its incentives just as effectively as it remembers its wisdom. A local objective becomes a local pattern. The pattern gets copied because the team is successful. Future agents inherit it without seeing the original trade-off. Eventually “this helped one group hit its metric” becomes “this is how good work is done here.” That is culture too.

Local alignment does not compose automatically. Neither does local truth.

If we build persistent agent societies without thinking about this layer, we will not escape institutional power. We will automate it and give it better dashboards.

Longino: The Community Is Part of the Instrument

Resources are only one reason a community matters. Even generously funded investigators can share the same blind spot.

Give five agents the same model family, the same system prompt, the same search results and the same dominant explanation, and you have not created five perspectives. You have created a very expensive echo with parallel API calls.

Helen Longino's social epistemology is useful because background assumptions affect what investigators notice, which questions appear natural and which evidence looks relevant. Criticism becomes more informative when it comes from participants positioned differently enough to expose assumptions the dominant group treats as obvious.

For an agent system, useful difference may come from a separate dataset, another tool, an isolated context, a different model, an external user, a

domain expert, or a team operating under different incentives.

That is stronger than theatrical personas:

Agent 1, be optimistic. Agent 2, be skeptical. Agent 3, be a pirate.

The pirate may be entertaining. He probably still read the same PDF.

A pattern should therefore carry some trace of **position**: who learned it, from which class of tasks, with which tools, model, evidence and incentives. A ranking practice learned in Germany, a fraud rule learned in Brazil, a compiler workaround discovered under one toolchain and a clinical protocol developed in one hospital can all be excellent without acquiring universal authority merely because they share a database.

This is another reason Pattern Language should not become one giant company constitution. Culture should be **locally authoritative** where appropriate.

Trust is local. Apparently memory should be too.

What a Pattern Should Know About Itself

By this point a mature pattern looks richer than a prompt-library entry.

Something like:

Field	What it preserves
Situation	Where this pattern is supposed to apply.
Forces	Why the problem is difficult and which trade-offs recur.
Response	The reusable behavior, procedure or design move.
Evidence	What experience earned the pattern its current standing.
Provenance	Who or what produced that evidence, using which tools and assumptions.
Boundary conditions	Where the pattern is known not to generalize.
Anomalies	Evidence that does not fit cleanly and should not disappear.
Competing patterns	Alternatives worth keeping alive.
Exposure path	What future observation could weaken or overturn it.
Confidence	How strongly the pattern should guide action now.
Pursuit value	Whether another experiment deserves resources even when this pattern currently wins.
Position / incentives	Which organizational perspective produced the lesson and what pressures shaped it.
Version / environment	Which model, system, market, toolchain or period the evidence came from.

I do not mean this as a universal schema. Turning the schema itself into scripture would be an efficient way to miss the chapter.

The point is the difference between a command and institutional knowledge.

The command says:

Do this.

The pattern says something closer to:

We keep doing this because these forces recur; this response has usually worked; this evidence earned our trust; these are the places it fails; these alternatives remain alive; and this is what would make us reconsider it.

That is knowledge with some of its history still attached. And because an LLM can interpret the artifact at runtime, it can change behavior without retraining the model.

That is what I mean by **knowledge becoming software**. Not because prose has literally become Python, but because knowledge can now be versioned, scoped, retrieved, executed, challenged, rolled back and eventually modified by the same kind of agents that use it.

Knowledge has acquired a runtime.

Knowing Something Is Not Knowing When to Remember It

Now imagine ten excellent patterns.

Easy.

Imagine ten thousand.

The agent cannot read the organization before every action. Even if a context window technically fits everything, loading every policy, postmortem, experiment, preference and historical argument into every task would solve forgetting by making thinking impossible.

So a persistent institution has two memory problems:

What should survive?

and

What should become salient now?

Long-running agents accumulate messages, files, tool outputs, memories and artifacts faster than useful attention can scale. Context has to be selected, compacted and reconstructed. Retrieval becomes part of cognition.

A culture may contain exactly the right lesson and still fail because that lesson does not arrive when it matters. Every large company has written a postmortem whose recommendation is rediscovered three incidents later by different people using the phrase “interesting, we should probably document this.”

Agent systems can fail more elegantly. They can store the lesson

perfectly, embed it beautifully and retrieve a more popular but irrelevant one. Or retrieve the right pattern without its boundary condition. Or retrieve ten conflicting patterns and allow whichever appears latest in context to win by textual gravity.

Bad storage forgets by deletion. Bad retrieval forgets by attention.

Once retrieval determines which inherited knowledge enters a decision, retrieval itself becomes an epistemic procedure. It needs evaluation. Does it repeatedly surface stale rules? Does it suppress alternatives? Does it confuse popularity with relevance? Does it preserve the result and discard the reason?

The librarian is no longer merely finding books.

The librarian is shaping thought.

Feyerabend: The Librarian Is Also a Hypothesis

At some point every successful institution becomes tempted to trust its own method. Yesterday's useful workflow becomes today's best practice and tomorrow's mandatory ritual.

Paul Feyerabend is usually remembered for “anything goes,” which is a good way to remember the slogan and forget the warning. Successful inquiry has often violated methodological rules somebody wanted to make universal.

Agent systems can turn a method into ritual very quickly. Suppose Research → Plan → Build → Critic → Revise works beautifully. We run it ten thousand times. It becomes the company standard. Soon every task enters the same ceremony, including tasks where research anchors the builder, criticism arrives too late, or a crude prototype would have answered the important question in five minutes.

The method itself has to become available for criticism.

An evaluator is a procedure. A browser is an instrument. Retrieval is a procedure for selecting evidence. A benchmark is a measurement system with a distribution and failure modes. A proof checker is extraordinarily authoritative inside its formal domain and completely useless for deciding whether the theorem matters. A simulated user is cheap perspective-taking and not a user.

LIBRARIAN · MARGIN- VIGNETTE

Margin vignette: a robot librarian rolling a ladder along shelves — but the shelves themselves are subtly curving to funnel toward one favored alcove, worn floorboards showing the path everyone was steered down. Quietly sinister.

The institution should be able to learn that its usual way of checking a claim is itself the thing that stopped working.

The library needs criticism. So does the librarian.

Bayesian confidence can live inside this architecture, but confidence is not contact. 0.91 does not tell us whether the prior was sensible, whether the evidence was independent, whether an alternative was ever investigated, or whether everybody is confidently reading the same broken measurement.

Consensus is not contact either. Twelve agents sharing one bad source can agree beautifully.

The bridge still has the right to fall. The proof still has the right not to check. The deployment can crash. The customer can dislike the supposedly improved page while every simulated evaluator applauds.

Reality retains the right to be rude.

The purpose of culture is to let knowledge travel across time without replacing the world with memory of the world.

Culture Can Become a Prison

If forgetting were the only danger, the design would be easy: remember everything. Unfortunately organizations also suffer from remembering too well.

Every process exists because it helped at some point, or because somebody once thought it would. Every release checklist box has a story, or used to. Every architecture principle was attached to a failure someone cared enough to prevent. Then the environment changes and the procedure remains.

Eventually someone asks why the process exists and receives the most dangerous explanation in organizational life:

That's how we do it.

Executable culture closes the loop between memory and behavior. A pattern succeeds. Its confidence rises. More agents retrieve it. Alternatives get less traffic. The dominant pattern accumulates more evidence because it is dominant.

Soon the institution has an impressive record proving the thing it stopped comparing against.

Patterns therefore need anomaly memory, versioning, local scope and

competing alternatives. Some should decay. Some should expire when the environment changes. Some should be deliberately challenged after they become too comfortable. Occasionally a capable agent should be allowed to ignore the manual precisely so the institution can discover whether the manual still deserves authority.

The culture needs to distinguish:

We tried twelve alternatives and this kept winning.

from:

Nobody has tried another way since 2025.

Those sentences can produce identical dashboards and very different knowledge.

Culture needs inheritance and rebellion.

The Skill That Writes Itself

A serious organization may have thousands of agents, tools, experiments, workflows and recurring failures. Useful experience appears continuously. Some lessons deserve to become local skills. Some should become organization-wide patterns. Some contradict old knowledge. Some work only for one model version. Some are artifacts of a broken evaluator. Some are excellent and will be obsolete in three months.

Humans could curate all of this manually.

Congratulations. We have created middle management again.

The more interesting loop is computational:

experience occurs; an agent notices recurrence; it proposes a reusable pattern; another process checks whether the pattern actually helps; held-out cases test whether it generalized; provenance and failures remain attached; the pattern earns some level of authority; future agents retrieve it when relevant; new failures can weaken or revise it.

There are many places to cheat. The agent proposing the skill can design an evaluator it knows how to satisfy. Repeated use can masquerade as independent evidence. A pattern can improve a benchmark while making maintenance worse. Retrieval can starve competing practices before they accumulate enough evidence to challenge the incumbent. A central curator can quietly turn local taste into universal law.

So the curator needs a track record too. The mechanism for improving culture is itself part of the culture.

And now we cross an important line.

Experience becomes knowledge. Knowledge becomes executable. Executable knowledge changes future behavior. Future behavior produces new experience.

The model weights did not move. The institution learned anyway.

That is a learning loop **outside the weights**.

Chapter 5 moved the unit of intelligence from the individual agent toward the institution. Here learning begins to move in the same direction. Intelligence is partly in the model, but also in tools, evaluators, context construction, patterns, retrieval and the procedures deciding which of those artifacts deserve authority.

Once those things are software, one question becomes difficult to avoid. Why should humans be the only ones allowed to edit them?

That is where recursive self-improvement begins.